

15.1: Structures

[This section corresponds to K&R Sec. 6.1]

The basic user-defined data type in C is the *structure*, or *struct*. (C structures are analogous to the *records* found in some other languages.) Defining structures is a two-step process: first you define a ``template" which describes the new type, then you declare variables having the new type (or functions returning the new type, etc.).

As a simple example, suppose we wanted to define our own type for representing complex numbers. (If you're blissfully ignorant of these beasts, a complex number consists of a ``real" and ``imaginary" part, where the imaginary part is some multiple of the square root of negative 1. You don't have to understand complex numbers to understand this example; you can think of the real and imaginary parts as the *x* and *y* coordinates of a point on a plane.) FORTRAN has a built-in complex type, but C does not. How might we add one? Since a complex number consists of a real and imaginary part, we need a way of holding both these quantities in one data type, and a structure will do just the trick. Here is how we might declare our complex type:

```
struct complex
{
    double real;
    double imag;
};
```

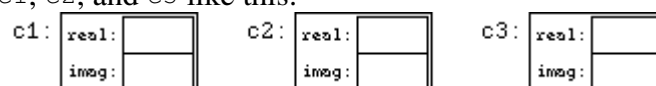
A structure declaration consists of up to four parts, of which we can see three in the example above. The first part is the keyword `struct` which indicates that we are talking about a structure. The second part is a name or *tag* by which this structure (that is, this new data type) will be known. The third part is a list of the structure's *members* (also called *components* or *fields*). This list is enclosed in braces {}, and contains what look like the declarations of ordinary variables. Each member has a name and a type, just like ordinary variables, but here we are not declaring variables; we are setting up the structure of the structure by defining the collection of data types which will make up the structure. Here we see that the `complex` structure will be made up of two members, both of type `double`, one named `real` and one named `imag`.

It's important to understand that what we've defined here is just the new data type; we have *not* yet declared any variables of this new type! The name `complex` (the second part of the structure declaration) is not the name of a variable; it's the name of the structure type. The names `real` and `imag` are not the names of variables; they're identifiers for the two components of the structure.

We declare variables of our new `complex` type with declarations like these:

```
struct complex c1;
or
struct complex c2, c3;
```

These look almost like our previous declarations of variables having basic types, except that instead of a type keyword like `int` or `double`, we have the two-word type name `struct complex`. The keyword `struct` indicates that we're talking about a structure, and the identifier `complex` is the name for the particular structure we're talking about. `c1`, `c2`, and `c3` will all be declared as variables of type `struct complex`; each one of them will have real and imaginary parts buried inside them. (We'll see how to get at those parts in the next section.) Using our graphic, ``labeled box" notation, we could draw representations of `c1`, `c2`, and `c3` like this:



Actually, these pictures are a bit misleading; the outer box indicating each composite structure suggests that there might be more inside them than just the two members, `real` and `imag` (that is, more than the two values of type `double`). A simpler but more representative picture would be:



The only memory allocated is for two values of type `double` (the two boxes); all the names are just for our convenience and the compiler's reference; none are typically stored in the program's memory at run time.

Notice that when we define structures in this way we have not quite defined a new type on a par with `int` or `double`. We can *not* say

```
complex c1;           /* WRONG */
```

The name `complex` does not become a full-fledged type name like `int` or `double`; it's just the name of a particular structure, and we must use the keyword `struct` *and* the name of a particular structure (e.g. `complex`) to talk about that structure type. (There is a way to define new full-fledged type names like `int` and `double`, and in C++ a new structure does automatically become a full-fledged type, but we won't worry about these wrinkles for now.)

I said that a structure definition consisted of up to four parts. We saw the first three of them in the first example; the fourth part of a full structure declaration is simply a list of variables, which are to be declared as having the structure type at the same time as the structure itself is defined. For example, if we had written

```
struct complex
{
    double real;
    double imag;
} c1, c2, c3;
```

we would have defined the type `struct complex`, and right away declared three variables `c1`, `c2`, and `c3` all of type `struct complex`.

In fact, three of the four parts of a structure declaration (all but the keyword `struct`) are optional. If a declaration contains the keyword `struct`, a structure tag, and a brace-enclosed list of members (as in the first structure definition we saw), it's a definition of the structure itself (that is, just the template). If a declaration contains the keyword `struct`, a structure tag, and a list of variable names (as in the first declarations of `c1`, `c2`, and `c3` we saw), it's a declaration of those variables having that structure type (the structure type itself must of course typically be declared elsewhere). If a declaration contains all four elements (as in the second declaration of `c1`, `c2`, and `c3` we saw), it's a definition of the structure type *and* a declaration of some variables. It's also possible to use the first, third, and fourth parts:

```
struct {
    double real;
    double imag;
} c1, c2, c3;
```

Here we declare `c1`, `c2`, and `c3` as having a structure type with no tag name, which is not usually very useful, because without a tag name we won't be able to declare any other variables or functions of this type for `c1`, `c2`, and `c3` to play with. (Finally, it's also possible to declare just the tag name,

leaving both the list of members and any declarations of variables for later, but this is only needed in certain fairly rare and obscure situations.)

Because a structure definition can also declare variables, it's important not to forget the semicolon at the end of a structure definition. If you accidentally write

```
struct complex
{
    double real;
    double imag;
}
```

without a semicolon, the compiler will keep looking for something later in the file and try to declare it as being of type `struct complex`, which will either result in a confusing error message or (if the compiler succeeds) a confusing misdeclaration.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1996-1999 // [mail](#) [feedback](#)